

## Coding Challenge

**Time Limit:** 30 Minutes | **Task:** Employee Task Tracker | **Total Marks:** 50

### Objective

Build a small Full Stack **Employee Task Tracker** where employee tasks can be created, viewed, updated, filtered, and deleted. Both frontend and backend are mandatory.

### Task Requirements

Create a form containing **Employee ID (Auto Generating)**, **Employee Name**, **Task Name**, and **Priority (Low / Medium / High)**. **Employee ID must be treated as the primary employee identifier and cannot be empty.** When a task is created, the backend must generate a unique Task ID and automatically set its status to **PENDING**; the frontend must not send the default status.

Display all tasks in a table as: **Employee ID , Employee Name , Task , Priority , Status, Action.**

Example: **EMP001, Rahul, Fix Login Bug , High , PENDING , Complete, Delete.**

### Backend & Status Logic

Implement **POST /tasks** to create a task, **GET /tasks** to retrieve all tasks, **PATCH /tasks/:id/status** to update task status, and **DELETE /tasks/:id** to delete a task. The allowed status flow is **PENDING to COMPLETED** only. The backend must validate status changes, reject invalid Task IDs/status values, and must not allow a **COMPLETED** task to return to **PENDING**.

### Frontend, Dashboard & Filter

Each pending task must provide **Mark Completed** and **Delete** actions. Mark Completed must call the PATCH API, and all changes must appear without manually refreshing the page. Completed

tasks should not display the Mark Completed action. Display live counts as **Total Tasks: X, Pending: X, Completed: X** and provide an **All, Pending, Completed** filter. Counts must automatically update after adding, completing, or deleting tasks; hardcoded values are not allowed.

## Validation

The backend must reject requests when **Employee Name is empty, Task Name is empty, or Priority is invalid**. Appropriate API error responses must be returned.

## GitHub Submission ( Mandatory)

Complete the application and push the working source code to a **new public GitHub repository within 30 minutes**. The repository must contain the complete source code, **.gitignore, README.md, and basic setup/run instructions**. Do not push **node\_modules, .env, passwords, credentials, API keys, or secrets**. Submit only the public GitHub repository URL in the provided google form, for example:

[\*https://github.com/username/employee-task-tracker\*](https://github.com/username/employee-task-tracker).

The latest working code must be pushed before the timer ends.

## Rules & Submission Requirements

Any Full Stack technology may be used. A database is mandatory, and all task data must be stored in the database; in-memory storage will not be accepted. The application must be fully functional, and the candidate must understand and be able to explain the submitted code.

After completing the challenge, take a full-screen screenshot of the working application with the complete browser/application window clearly visible. Upload the screenshot to the submission form shared in the group.

The complete source code must also be pushed to a public GitHub repository, and the GitHub repository URL must be submitted through the same form before the deadline.